

Sesiunea 22

Memorator

Algoritmi populari

- Verificare daca un numar este prim
- Aflarea divizorilor proprii si improprii ai unui numar
- Parcurgerea unui numar cifra cu cifra de la dreapta la stanga (de la sfarsit la inceput)
- Parcurgerea unui numar cifra cu cifra de la stanga la dreapta (de la inceput la sfarsit)
- Aflarea inversului unui numar
- Verifica daca un numar este palindrom
- Sortarea unui vector - bubblesort
- Impartirea in factori primi ai unui numar
- Cel mai mare divizor comun (cmmdc => gcd in engleza => algoritmul lui euclid)
 - Iti recomand sa stii cel putin una dintre cele 3 forme ale algoritmului lui euclid
 - Vezi aici: https://ro.wikipedia.org/wiki/Algoritmul_lui_Euclid
- Cel mai mic multiplu comun (cmmm)

Vectori

- Sortarea unui vector
- Adaugi intr-un vector la o anumite pozitie
- Sa stergi din vector
- Vectorul de frecventa / Vector de aparitii

Siruri de caractere

- Intelegere profunda a functiilor elementare
 - `strcpy`
 - `strncpy`
 - `strcat`
 - `strncat`
 - `strcmp`
 - `strncmp`

```
#include <iostream>
using namespace std;
void numaraCifreleImpare(int &n);
int main() {
    char s1[20] = "Alexandru";
    char s2[20] = "Alexandra";
    if (strncmp(s1, s2, 4) == 0) {
        cout << "S1 = S2";
    } else {
        cout << "S1 <> S2";
    }
}
```

```
    }
    return 0;
}
```

- `strtok`

```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char s1[20] = "Ana are;ac cu_ata";
    char * cuvant = strtok(s1, " ;_");
    while (cuvant != NULL) {
        cout << cuvant << endl;
        cuvant = strtok(NULL, " ;_");
    }
    return 0;
}
```

- `strchr`
- `strstr`
- `strlen`

- Matrici de cuvinte

```
#include <iostream>
using namespace std;
int main() {
    char matriceCuvinte[10][20];
    for (int i = 0; i < 10; i++) {
        cin >> matriceCuvinte[i];
    }

    for (int i = 0; i < 10; i++) {
        cout << matriceCuvinte[i] << endl;
    }
    return 0;
}
```

- Verificare daca un caracter este sau nu vocala
- Verificare daca un caracter este sau nu consoana
- Verificare daca un caracter este sau nu numar
 - prin folosirea functiei din `cctype`
 - folosind functiile elementare de mai sus

- Convertesti un caracter intr-un numar
 - De exemplu din '5' sa obtii 5;

Functii / Subprograme

- Functii care intorc valori

```
#include <iostream>
using namespace std;
int sum(int[], int);
int main() {
    int numere[] = {1,2,4,45,5,6};
    int numarElemente = sizeof(numere) / sizeof(numere[0]);
    // Apelul functiei
    cout << sum(numere, numarElemente) << endl;
    return 0;
}

// Definirea functiei
int sum(int arr[], int n) {
    int suma = 0;
    for (int i = 0; i < n; i++) {
        suma += arr[i];
    }
    return suma;
}
```

- Functii care nu intorc valori

```
#include <iostream>
using namespace std;
void afiseazaSuma(int arr[], int n);
int main() {
    int numere[] = {1,2,4,45,5,6};
    int numarElemente = sizeof(numere) / sizeof(numere[0]);
    // Apelul functiei
    afiseazaSuma(numere, numarElemente);
    return 0;
}

void afiseazaSuma(int arr[], int n) {
    int suma = 0;
    for (int i = 0; i < n; i++) {
        suma += arr[i];
    }
    cout << "Suma elementelor este " << suma;
}
```

- Functii care intorc valori prin parametri

```
#include <iostream>
using namespace std;
void numaraCifreleImpare(int &n);
int main() {
    int x = 1234532567;
    numaraCifreleImpare(x);
    cout << x << endl;
    return 0;
}

void numaraCifreleImpare(int &n) {
    int contor = 0;
    while (n > 0) {
        int ultimaCifra = n % 10;
        if (ultimaCifra % 2 == 1) {
            contor++;
        }
        n = n / 10;
    }
    n = contor;
}
```

- Sa faci diferente dintre definirea unei functii si apelul unei functii

```
#include <iostream>
using namespace std;
int sum(int[], int);
int main() {
    int numere[] = {1,2,4,45,5,6};
    int numarElemente = sizeof(numere) / sizeof(numere[0]);
    // Apelul functiei
    cout << sum(numere, numarElemente) << endl;
    return 0;
}

// Definirea functiei
int sum(int arr[], int n) {
    int suma = 0;
    for (int i = 0; i < n; i++) {
        suma += arr[i];
    }
    return suma;
}
```

- Lucrul cu functii recursive
- Lucrul cu functii recursive care intorc valori prin parametri

- Vezi rezolvarea de la S1 - Simulare 2026

Matrici

- Matrici patratice
 - Formula diagonala principala
 - Formula diagonala secundara
 - Aflare elemente pozitionate la intersectia celor 2 diagonale
 - Nord
 - Sud
 - Est
 - Vest
- Matrici oarecare
- Afisare matrice

Structuri

- Definire structura
- Definire vector de structuri
- Initializare variabila de tipul struct
- Definire structura care are ca membru o alta structura
- Diverse exercitii cu structuri

Grafuri

- Orientate vs Neorientate
- Graf hamiltonian
- Graf eulerian
- Arbore
- Subgraf vs Graf Partial vs Componenta Conexa vs Componenta tare conexa
- Diverse formule
 - Numarul de muchii intr-un arbore
 - Numarul de subgrafuri posibile
 - etc
- Vectorul de tati
- Lista de adiacenta
- Matricea de adiacenta

Pseudocod

- Echivalenta structurilor repetitive
- Definire si folosire a structurilor repetitive
 - De exemplu sa ai un for care pleaca de la n la 0 si cu pas diferit de 1 (gen din 3 in 3)
- De retinut
 - Atunci cand transformam din structura repetitiva cu test initial in structura repetitiva cu test final, neaparat trebuie sa punem o instructiune **daca** inainte de inceputul acesteia, altfel cele doua nu sunt echivalente.
 - De exemplu se da urmatorul pseudocod

```

citește m,n
(numere naturale nenule,  $m \leq n$ )
nr←-0; i←-m
repetă
  x←-1
  cât timp x*x<i execută
    x←-x+1
  ■
  dacă x*x=i atunci nr←-i
  altfel i←-i+1
  ■
până când i>n sau nr≠0
scrie nr

```

- Si se cere sa se inlocuiasca structura cât timp...execută cu o structură repetitivă cu test final
- Atentie mare care este structura ce trebuie inlocuita
- Dupa care, rezultatul corect ar fi asa:

```

citește m,n
(numere naturale nenule,  $m \leq n$ )
nr←-0; i←-m
repetă
  x←-1
  dacă x*x<i atunci
    execută
      x←-x+1
      ■cât timp x*x<i
  ■
  dacă x*x=i atunci nr←-i
  altfel i←-i+1
  ■
până când i>n sau nr≠0
scrie nr

```

- DAAAR un rezultat invalid ar fi:

```

citește m,n
(numere naturale nenule,  $m \leq n$ )
nr←-0; i←-m
repetă
  x←-1
  execută
    x←-x+1
    ■cât timp x*x<i
  ■
  dacă x*x=i atunci nr←-i
  altfel i←-i+1
  ■

```

```
    Lpână când i>n sau nr≠0  
    scrie nr
```

- Ar fi invalid deoarece instrucțiunea `executa...cat timp` va fi executată și dacă x va fi un număr care nu respecta condiția inițial (`x*x<i`)

Lucrul cu fișiere

- Citire din fișiere

```
#include <iostream>  
#include <fstream>  
using namespace std;  
  
int main() {  
    ifstream fin("bac.txt");  
    int numar, suma = 0;  
    while (fin >> numar) {  
        suma += numar;  
    }  
  
    cout << suma << endl;  
    return 0;  
}
```

- Scriere în fișiere
 - similar cu ce avem ca exemplu mai jos
- Adăugare în fișier (nu suprascrie ce deja există în fișier)

```
#include <iostream>  
#include <fstream>  
using namespace std;  
  
int main() {  
    cout << "Hello world!" << endl;  
    ifstream fin("bac.txt");  
    // A se nota parametrul ios::app  
    ofstream fout("bac.txt", ios::app);  
  
    int numar, suma = 0;  
    while (fin >> numar) {  
        suma += numar;  
    }  
  
    fout << suma << endl;  
    return 0;  
}
```